S03 – Introducción a BigQuery y SQL en la nube

Evidencias de ejecución – IT Academy, Data Analytics

Nombre	*Carlos Hisamitsu*
P2P	*Gabriela*
Fecha	*13/10/2026*
Repositorio GitHub	https://github.com/chisamitsu/ita-dataanalytics/tree/main/03-s03-bigquery-sql-nube

NIVEL 1: Entorno e ingesta hibrida (Code-First)

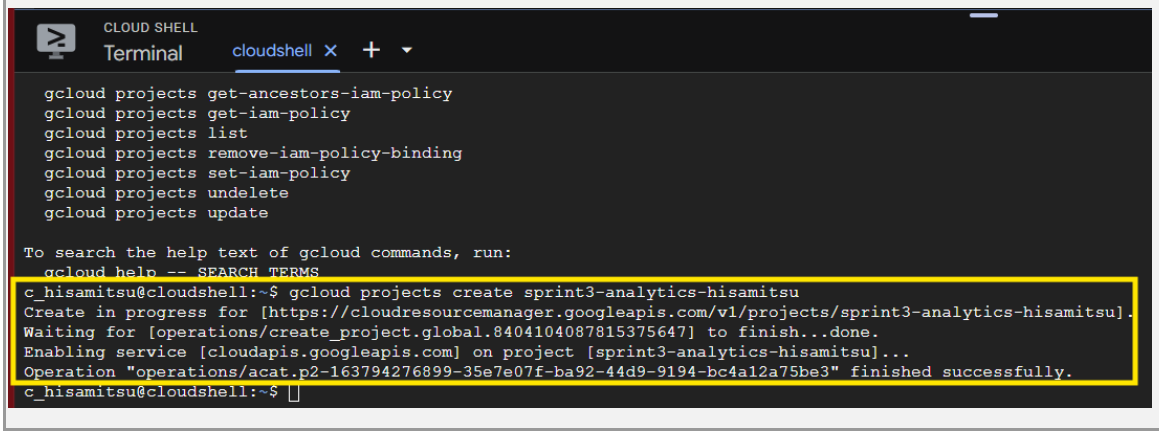
Ejercicio 1 – Arquitectura de datos (lógica vs. física)

Enunciado

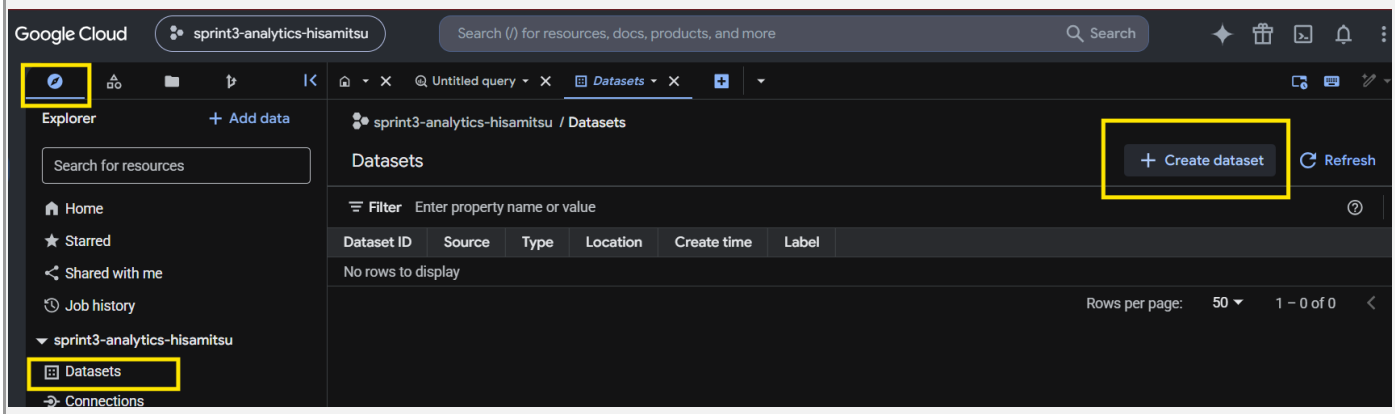
Crear el proyecto sprint3-analytics-[tu-nombre] y materializar la arquitectura Medallion con tres datasets (sprint3_bronze, sprint3_silver, sprint3_gold), cada uno creado con un método distinto: UI, SQL (CREATE SCHEMA) y Cloud Shell (bq).

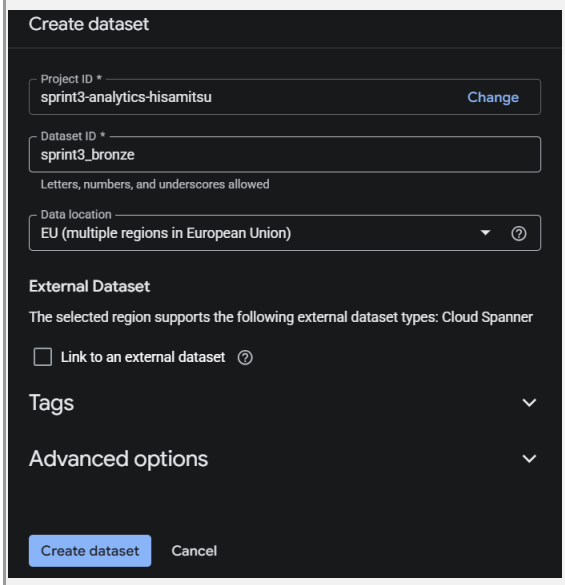
Evidencia de ejecución

Creación del Proyecto **sprint3-analytics-hisamitsu**

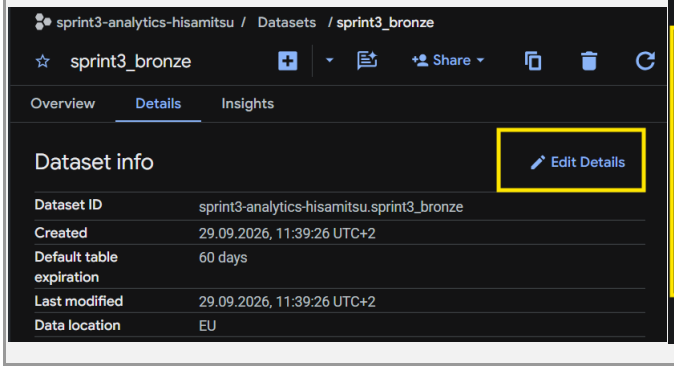


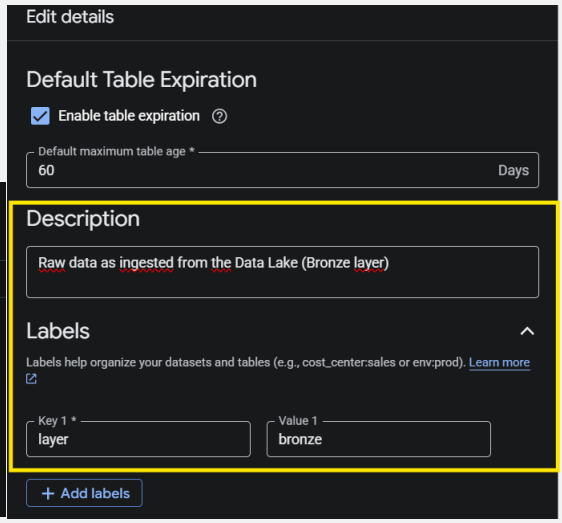
Creación del dataset **sprint3_bronze** (UI)



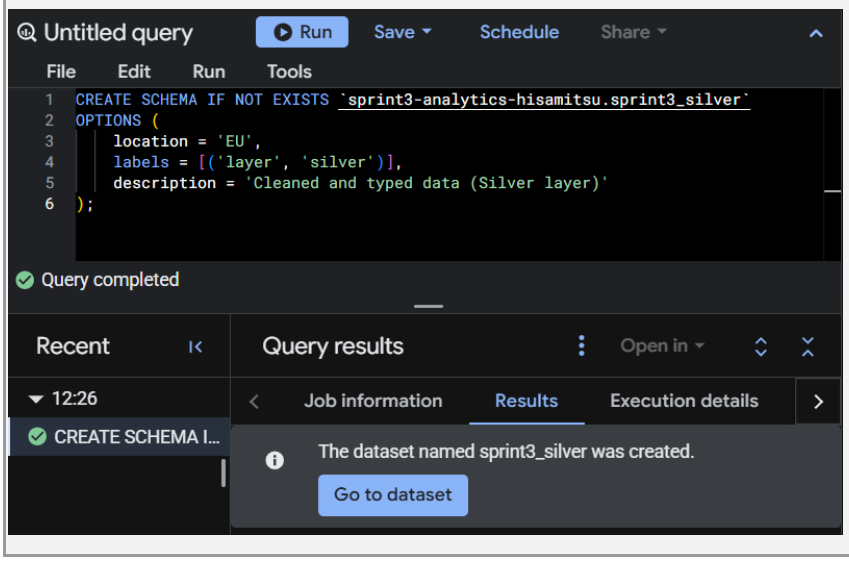


No permite añadir label ni description al crear un dataset desde la UI (ref); los he añadido después editando el dataset ya creado.

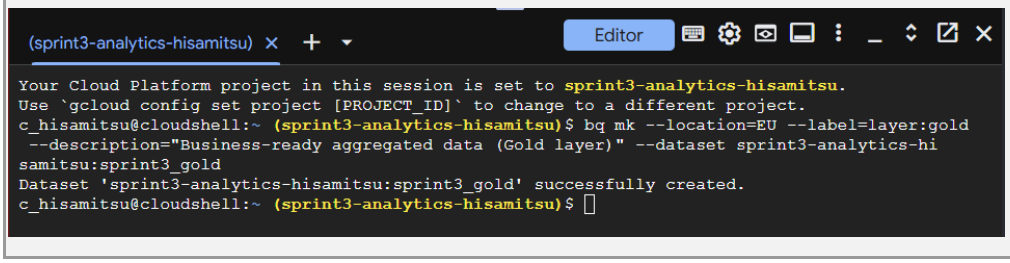




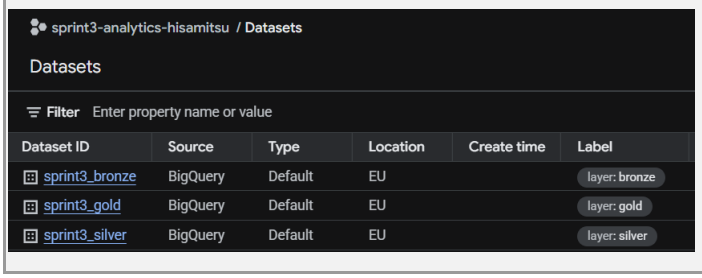
Creación del dataset **sprint3_silver** (SQL)



Creación del dataset **sprint3_gold** (bq)

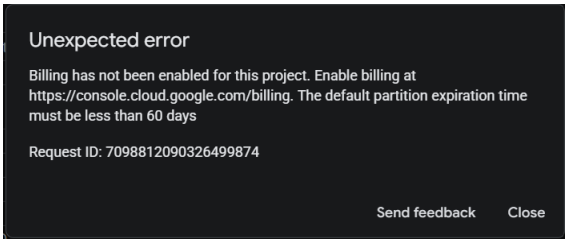


Los 3 datasets:



Comentarios sobre los resultados

Los pasos para añadir label y description al dataset sprint3_bronze desde la UI están descritos arriba, pero al intentar guardar los cambios me dio un error (el proyecto no tiene una cuenta de facturación asociada).



Al final los he actualizado con `bq update --set_label layer:bronze --description "..." sprint3-analytics-hisamitsu:sprint3_bronze.`

Ejercicio 2 – Ingesta en capa Bronze (tablas externas)

Enunciado

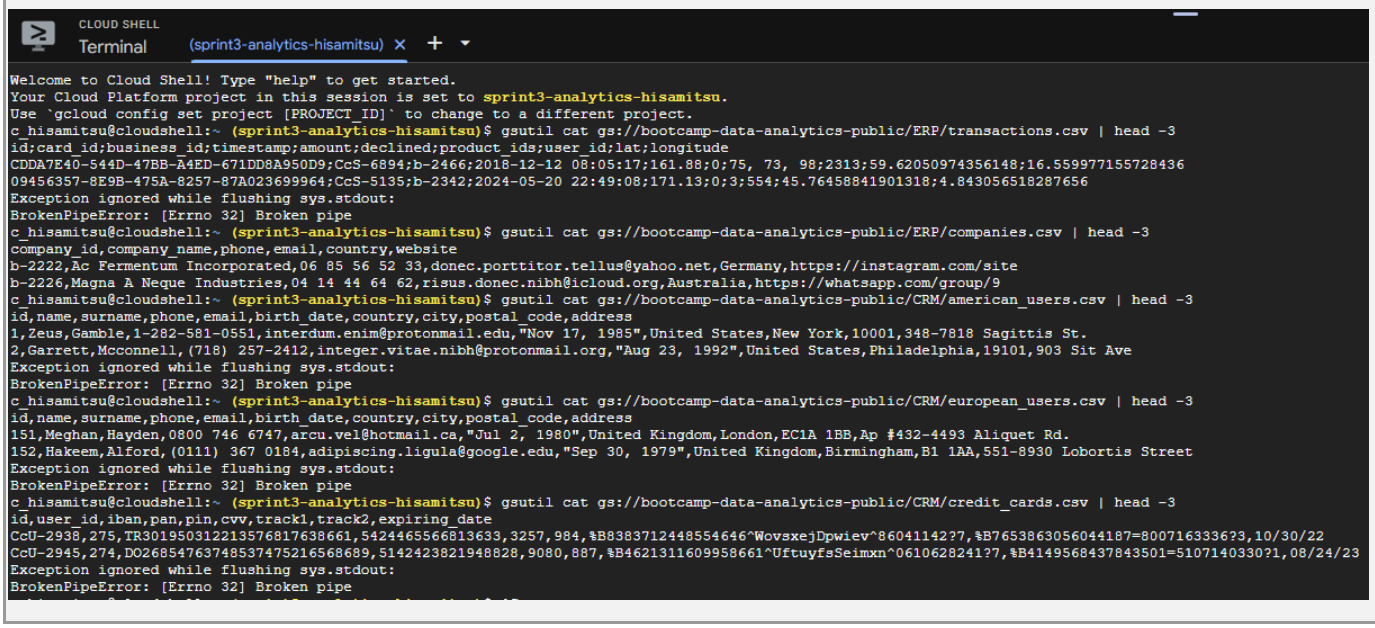
Crear las 5 tablas externas en sprint3_bronze apuntando al Data Lake: transactions_raw, companies_raw (define el esquema manualmente), american_users_raw, european_users_raw y credit_cards_raw.

Evidencia de ejecución

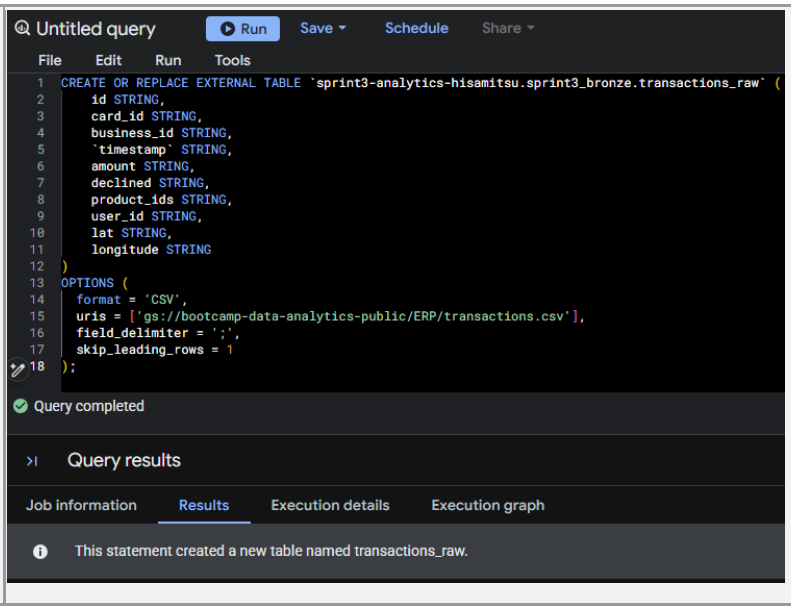
Nota: la primera vez usé autodetect en 4 de las 5 tablas. Lo rehice con esquema manual, todo STRING: Bronze captura los datos tal cual, sin tipar nada. El tipado se hace en Silver.

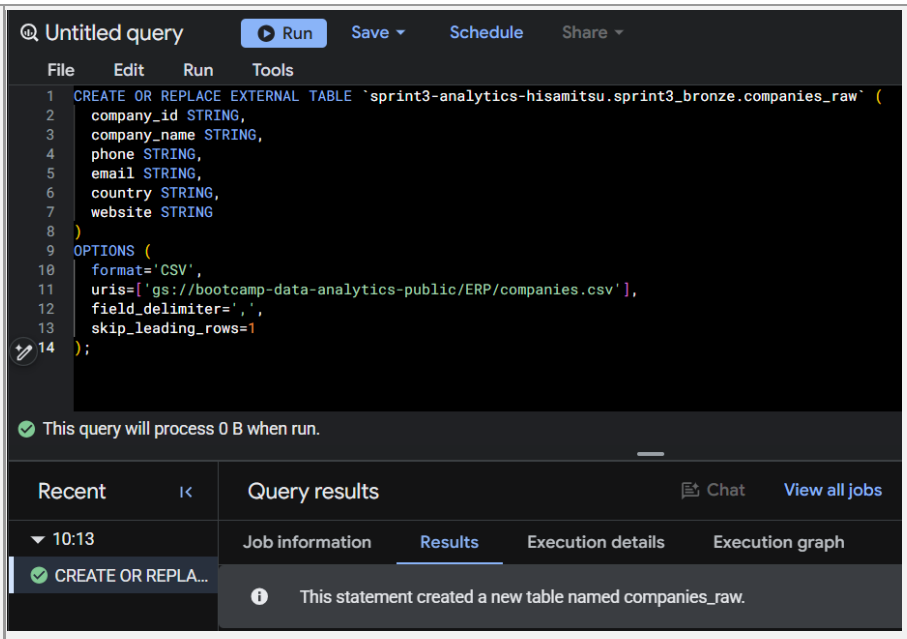
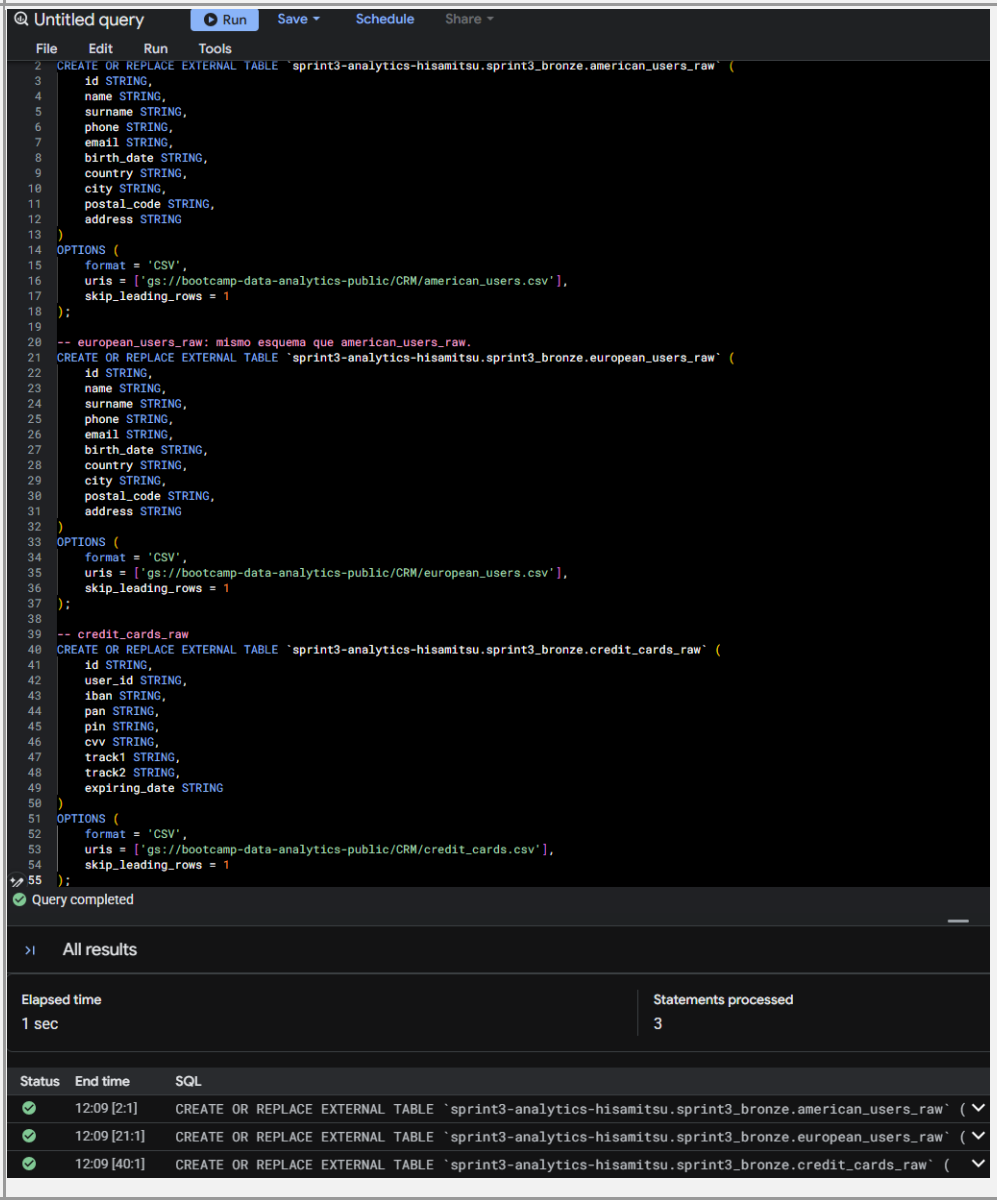
Para encontrar las columnas de cada archivo, usé el terminal

```
gsutil cat gs://bootcamp-data-analytics-public/ERP/transactions.csv | head -3
```

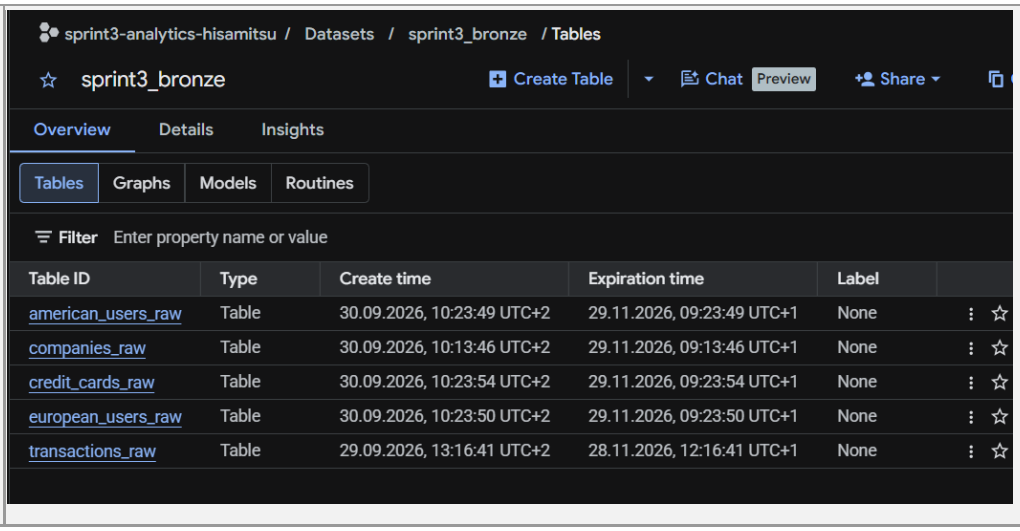


transactions_raw



companies_raw	
american_users_raw, european_users_raw, credit_cards_raw	

Tablas creadas

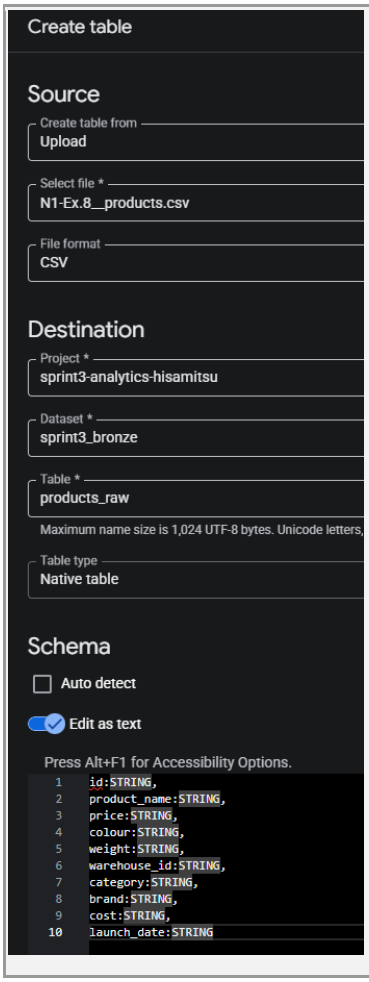


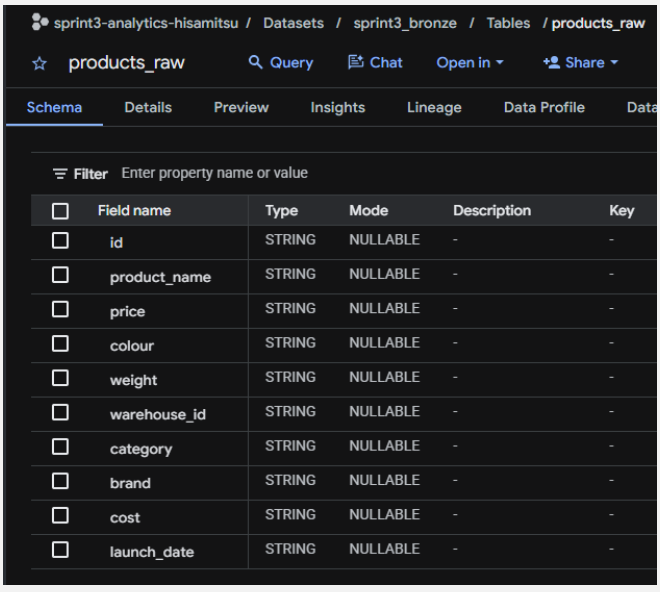
Ejercicio 3 – Carga de datos locales (Upload)

Enunciado

Subir products.csv (recursos del sprint 2) como tabla nativa products_raw en sprint3_bronze mediante Upload desde la UI.

Evidencia de ejecución





Ejercicio 4 – Arquitectura y rendimiento (asistido por IA)

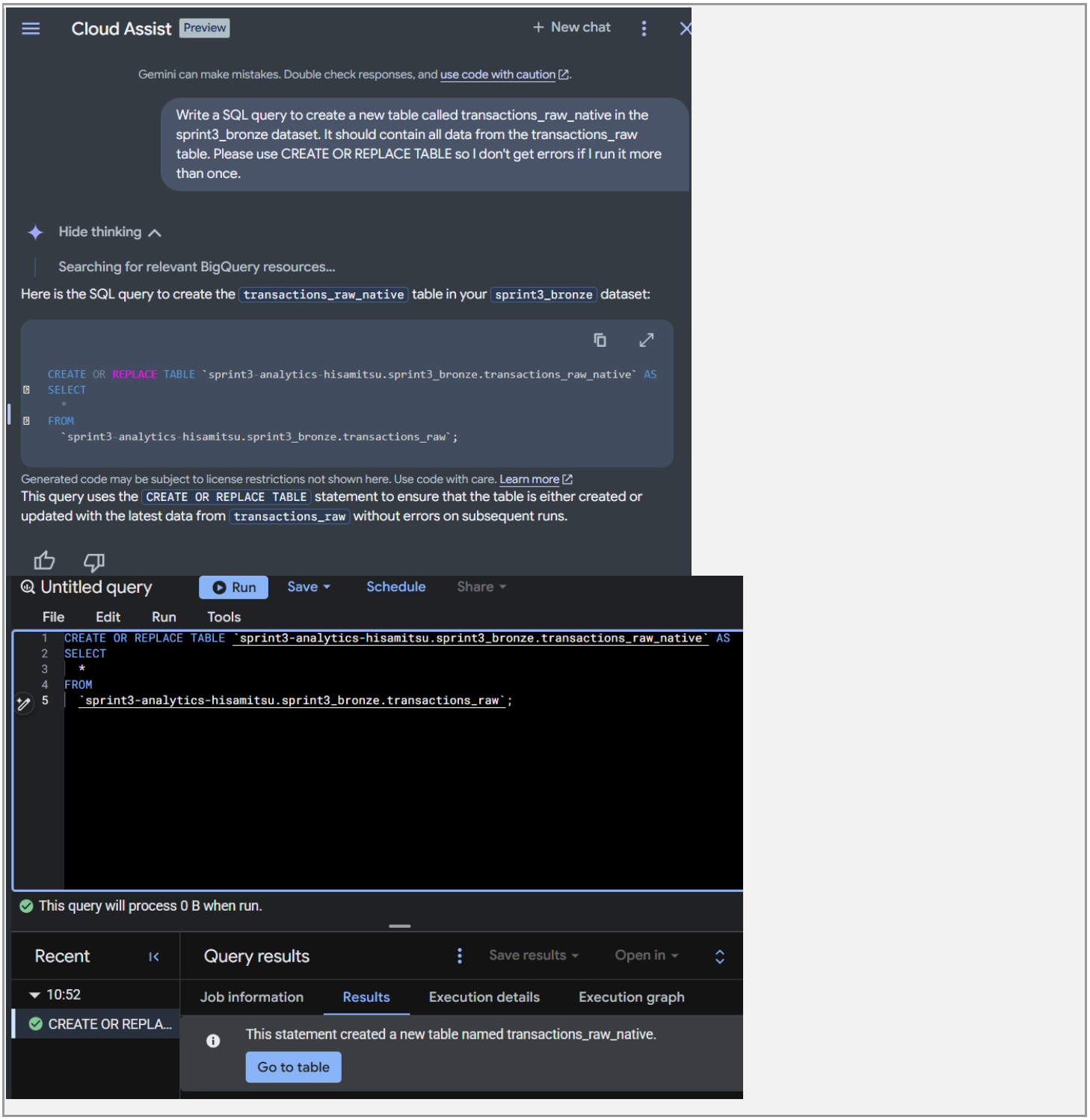
Enunciado

a) Crear transactions_raw_native como copia nativa de transactions_raw usando el asistente de IA de BigQuery.

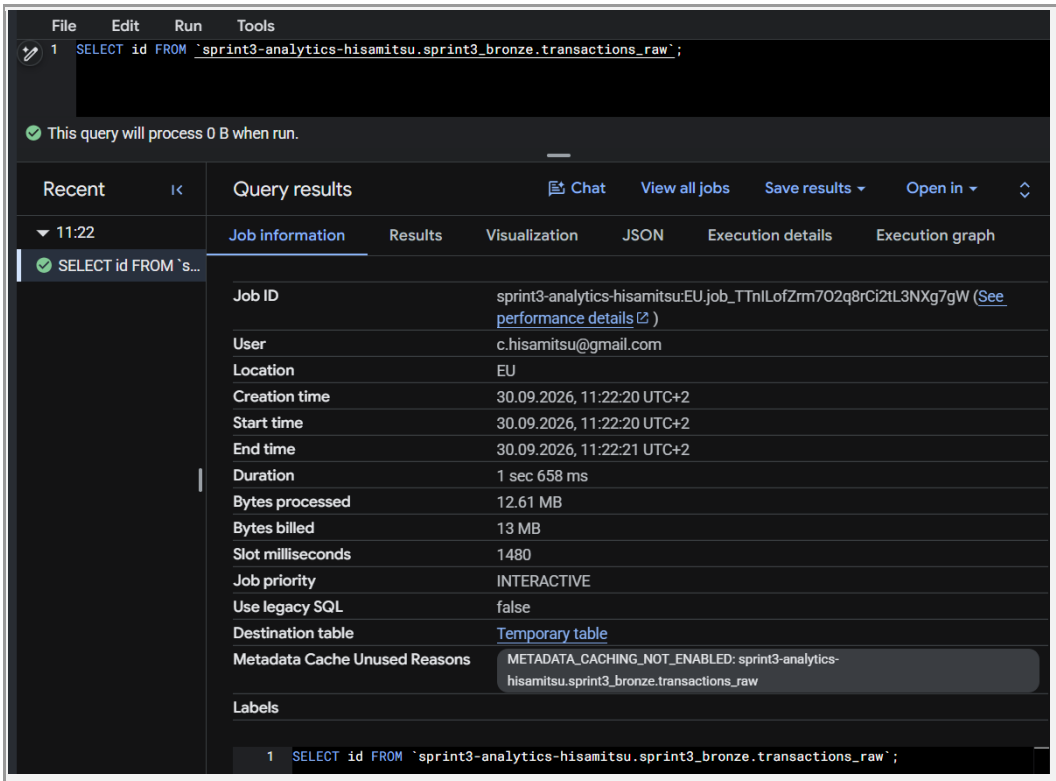
b) Comparar bytes procesados al leer una columna en la tabla externa vs. la nativa.

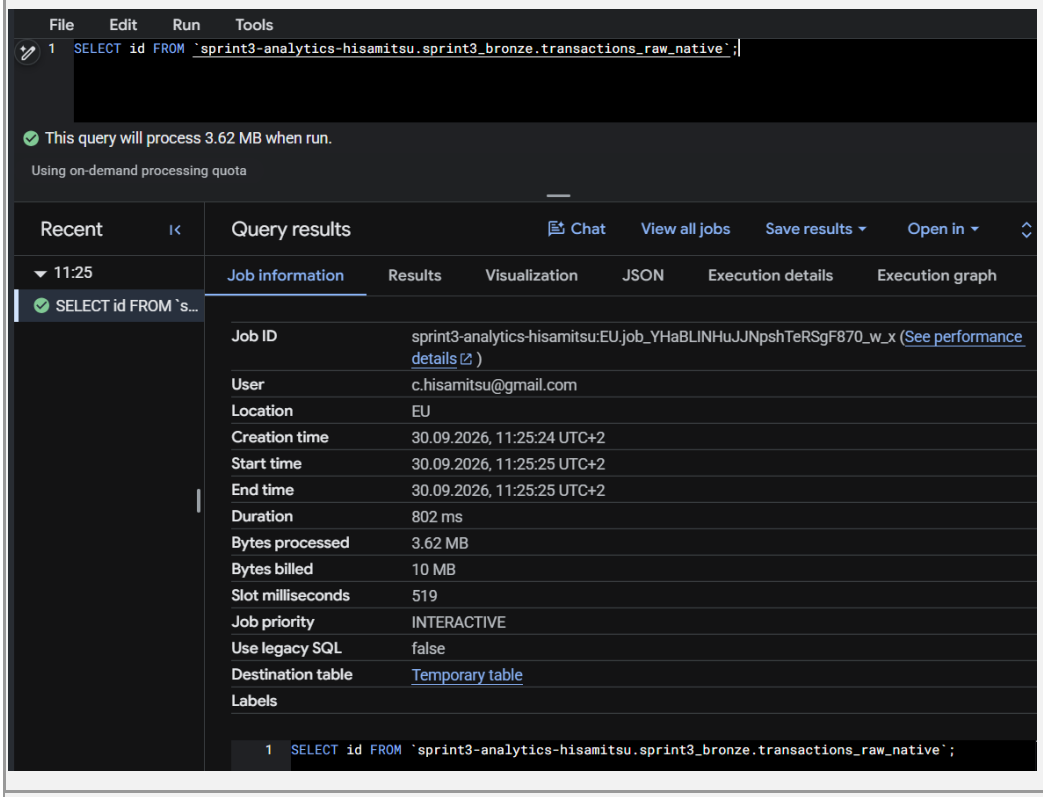
c) Comprobar si LIMIT reduce los bytes procesados.

a) Materialización asistida por IA



b) Auditoria de costes





El dry run no puede estimar el coste de una tabla externa y devuelve 0 bytes; la ejecución real procesa 12.61 MB. La tabla nativa sí estima bien y procesa solo 3.62 MB, por el almacenamiento columnar.

c) El peligro del LIMIT

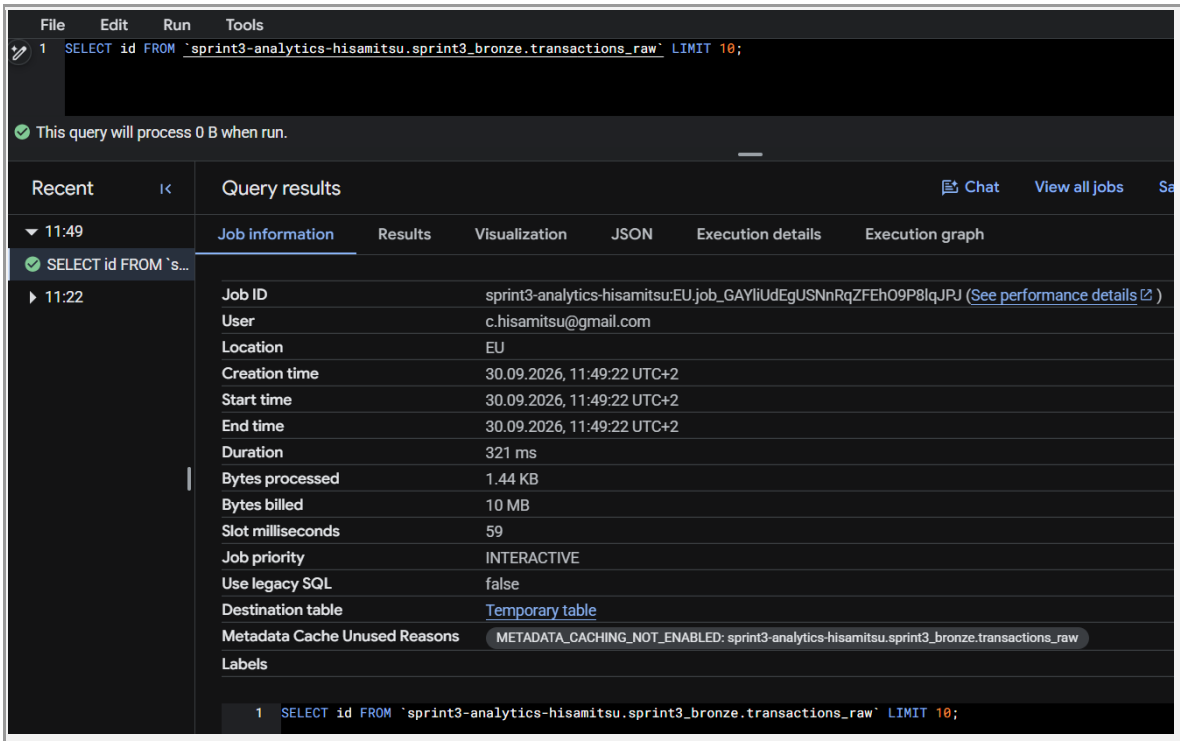


Tabla externa con LIMIT: **1.44KB**

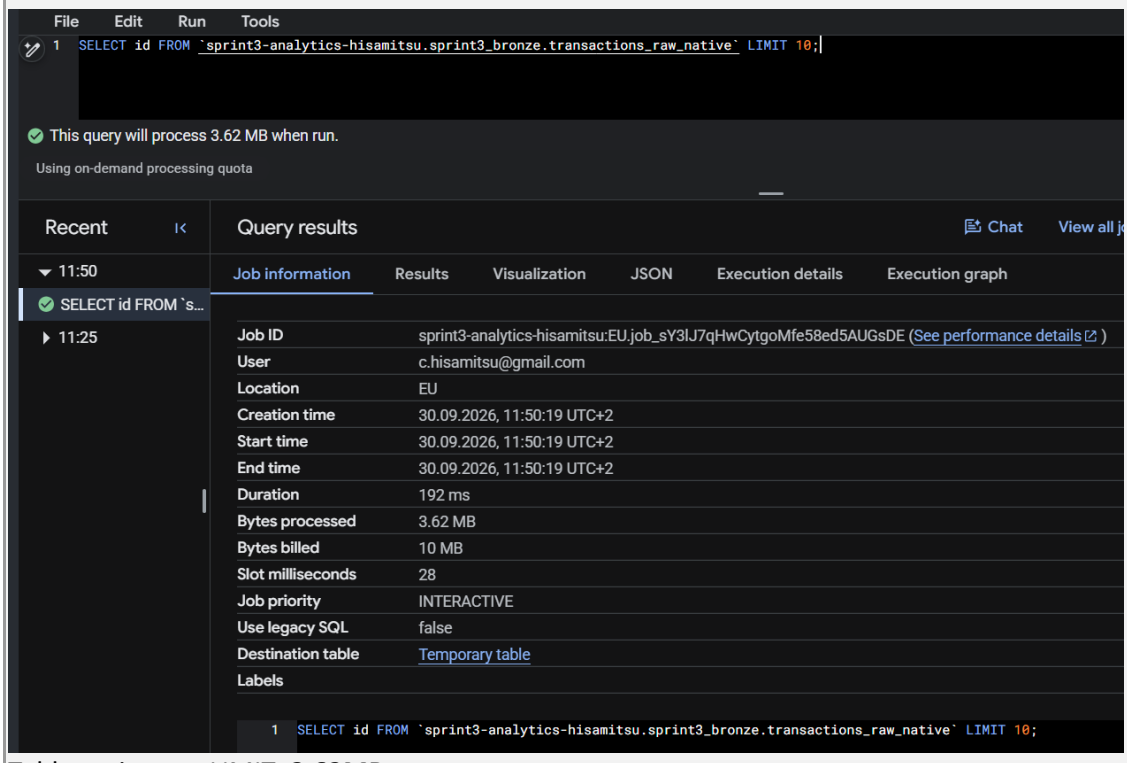


Tabla nativa con LIMIT: **3.62MB**

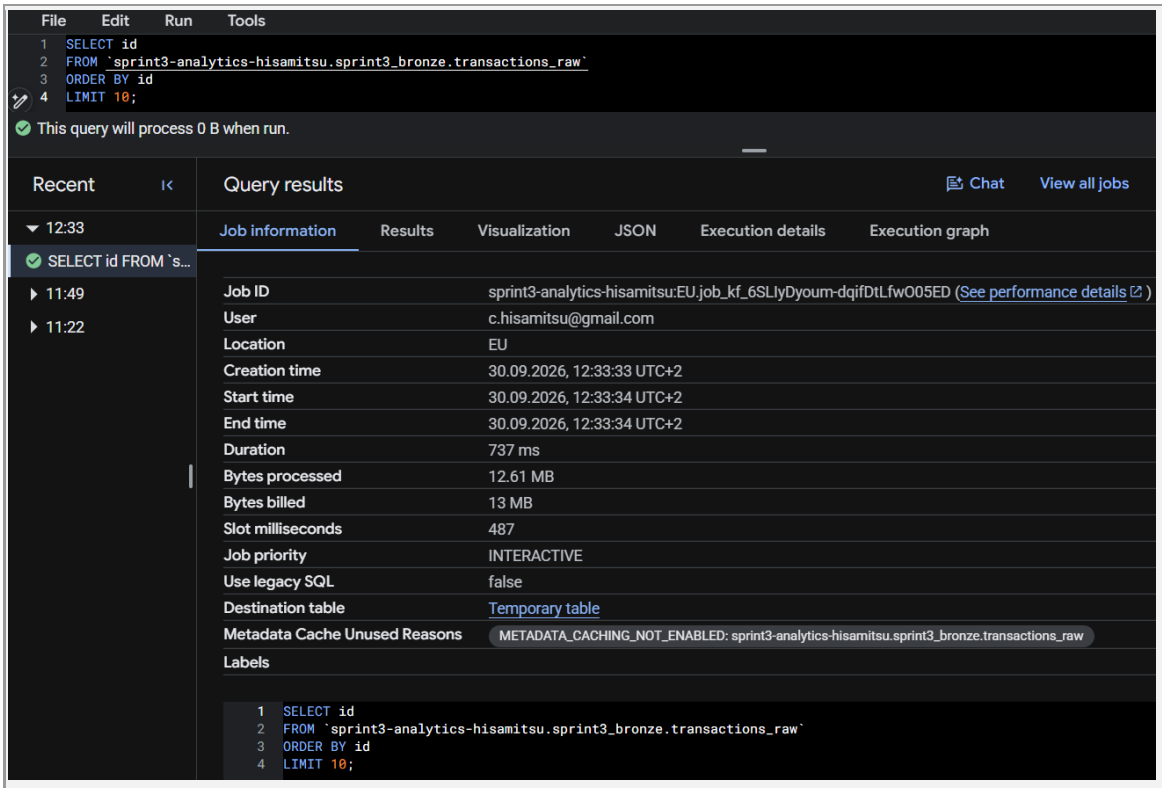


Tabla externa con LIMIT y ORDER BY: **12.61MB**

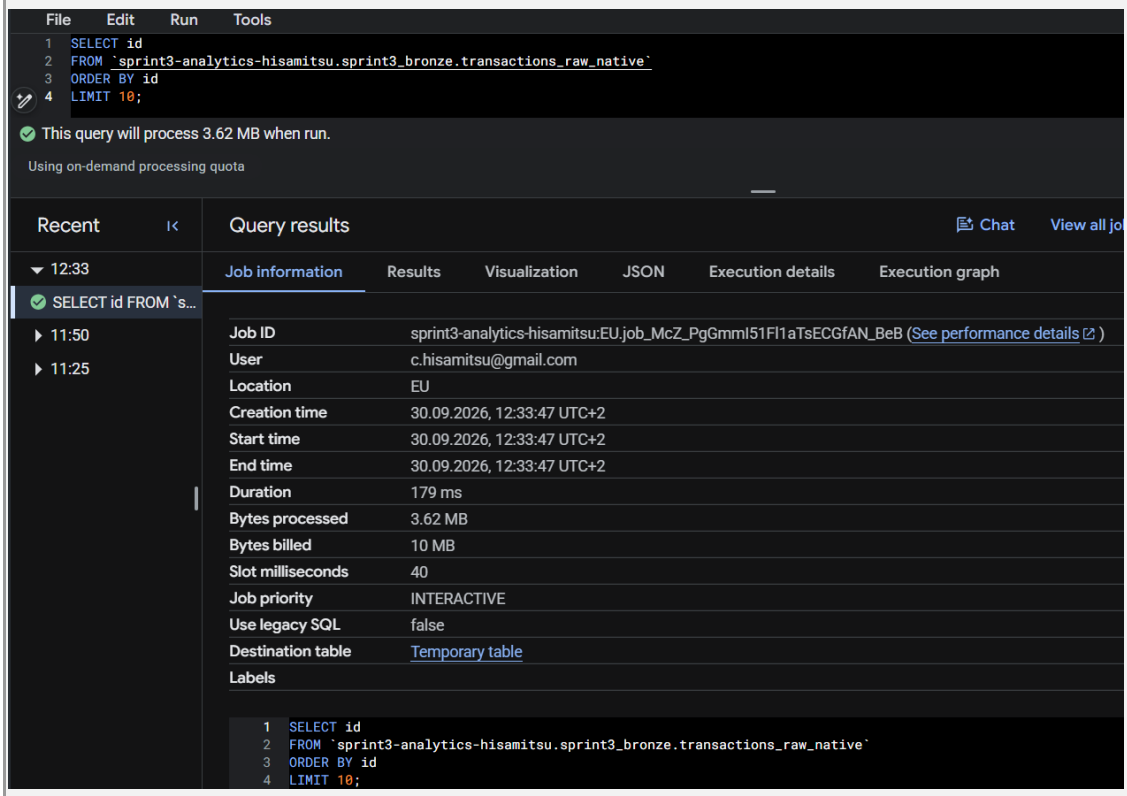


Tabla nativa con LIMIT y ORDER BY: **3.62MB**

Comentarios sobre los resultados

LIMIT no baja los bytes leídos en tablas no clusterizadas. Lo confirmo con transactions_raw_native: 3.62 MB con y sin LIMIT.

Con transactions_raw (externa) sí bajó, de 12.61 MB a 1.44 KB. Pero con ORDER BY id volvió a 12.61 MB. El lector del CSV solo puede parar en 10 filas si no hay que ordenar, filtrar o agregar.

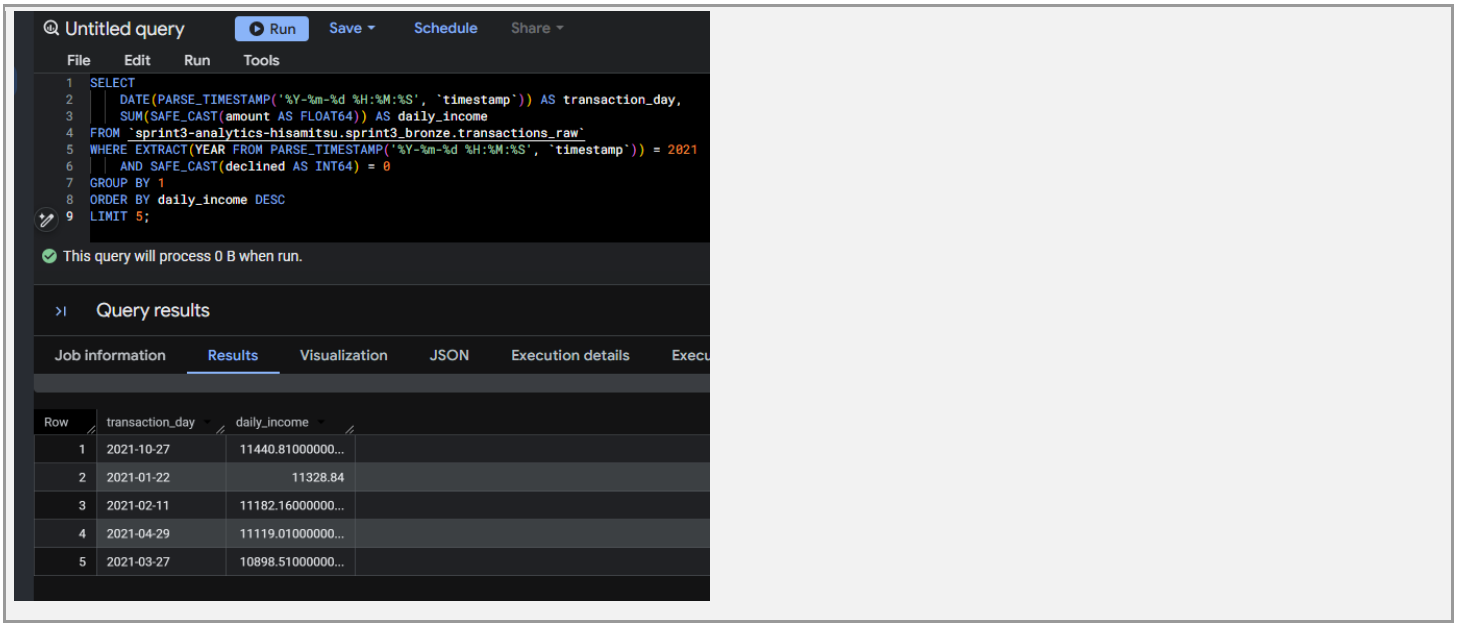
LIMIT no ahorra coste en ningún caso. Solo corta el resultado final, no la lectura.

Ejercicio 5 – Adaptación de sintaxis (reporting)

Enunciado

Los 5 días con más ingresos de 2021, parseando el campo timestamp (STRING) a fecha.

Evidencia de ejecución



Comentarios sobre los resultados

El campo timestamp llega como STRING en transactions_raw, así que lo parseo con `PARSE_TIMESTAMP('%Y-%m-%d %H:%M:%S', timestamp)` antes de aplicar DATE()/EXTRACT().

amount también es STRING, de ahí el `SAFE_CAST` a `FLOAT64` (para un mejor resultado, podría usar ROUND para mantener 2 dígitos).

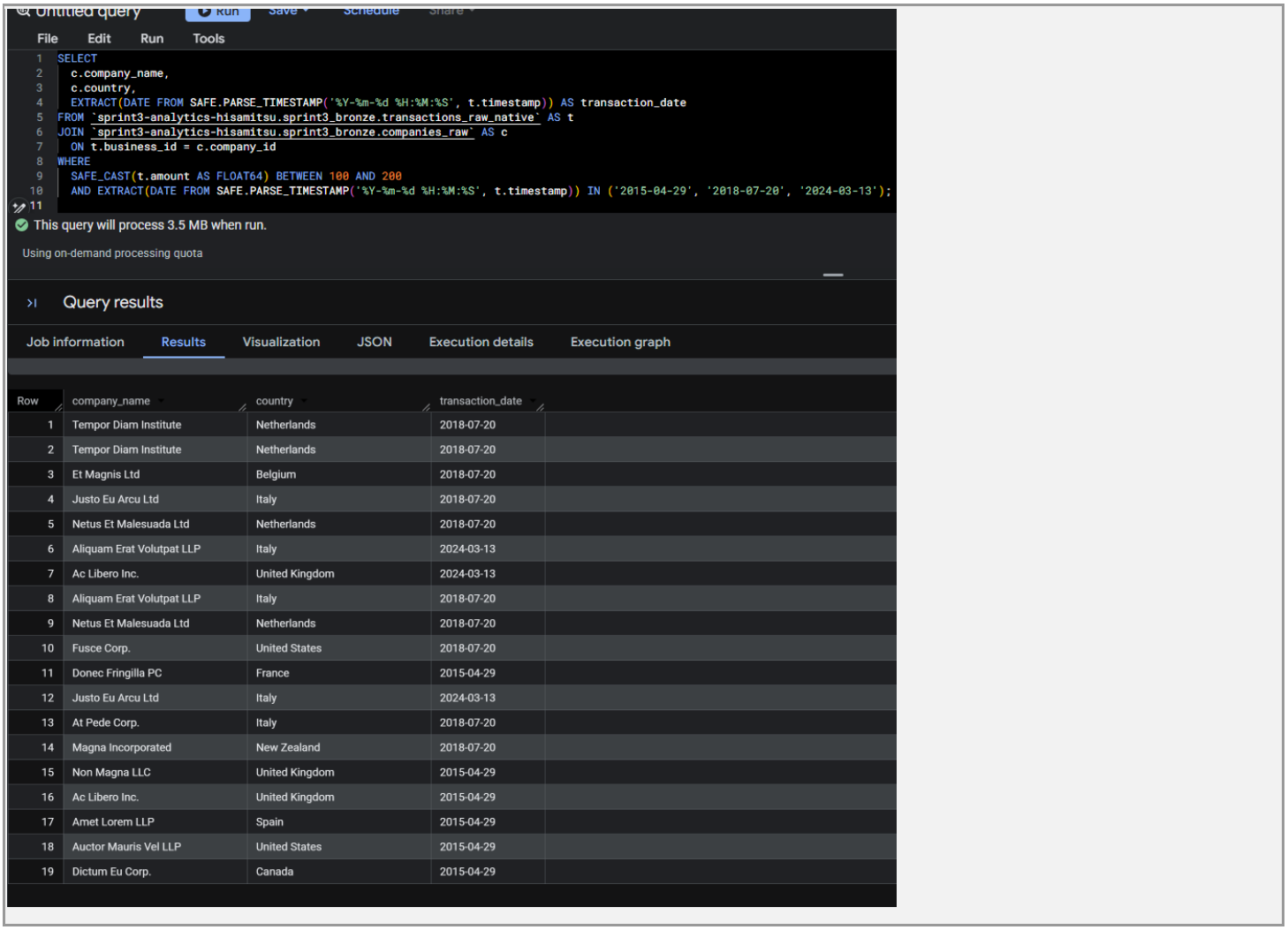
Añadí `declined = 0` porque "ingresos" no debería contar transacciones rechazadas (mismo criterio usado en Sprint02 para las consultas de ventas).

Ejercicio 6 – Consultas complejas

Enunciado

Nombre, país y fecha de las transacciones de compañías que operaron entre 100 y 200 euros en alguna de estas fechas: 29-04-2015, 20-07-2018 o 13-03-2024.

Evidencia de ejecución



Row	company_name	country	transaction_date
1	Tempor Diam Institute	Netherlands	2018-07-20
2	Tempor Diam Institute	Netherlands	2018-07-20
3	Et Magnis Ltd	Belgium	2018-07-20
4	Justo Eu Arcu Ltd	Italy	2018-07-20
5	Netus Et Malesuada Ltd	Netherlands	2018-07-20
6	Aliquam Erat Volutpat LLP	Italy	2024-03-13
7	Ac Libero Inc.	United Kingdom	2024-03-13
8	Aliquam Erat Volutpat LLP	Italy	2018-07-20
9	Netus Et Malesuada Ltd	Netherlands	2018-07-20
10	Fusce Corp.	United States	2018-07-20
11	Donec Fringilla PC	France	2015-04-29
12	Justo Eu Arcu Ltd	Italy	2024-03-13
13	At Pede Corp.	Italy	2018-07-20
14	Magna Incorporated	New Zealand	2018-07-20
15	Non Magna LLC	United Kingdom	2015-04-29
16	Ac Libero Inc.	United Kingdom	2015-04-29
17	Amet Lorem LLP	Spain	2015-04-29
18	Auctor Mauris Vel LLP	United States	2015-04-29
19	Dictum Eu Corp.	Canada	2015-04-29

Comentarios sobre los resultados

Esta query se parece a la del Ejercicio 2 de Sprint02: un JOIN entre transacciones y compañías para filtrar por datos de las dos tablas juntas.

Para escribir la query le pedí a Gemini: "we need a report that crosses data. Task: Lists the name, country and date of the transactions carried out by companies that carried out transactions between 100 and 200 euros on any of these dates: 29-04-2015, 20-07-2018 or 13-03-2024.".



Gemini metió declined = 'false' en su respuesta, pero lo quité. El enunciado pide transacciones realizadas, no ventas, y además ese filtro ni siquiera funcionaría: declined en transactions_raw es STRING con valores '0'/'1', no 'true'/'false'.

Dejé transactions_raw_native (lo que sugirió Gemini) en vez de transactions_raw. Mismo resultado.

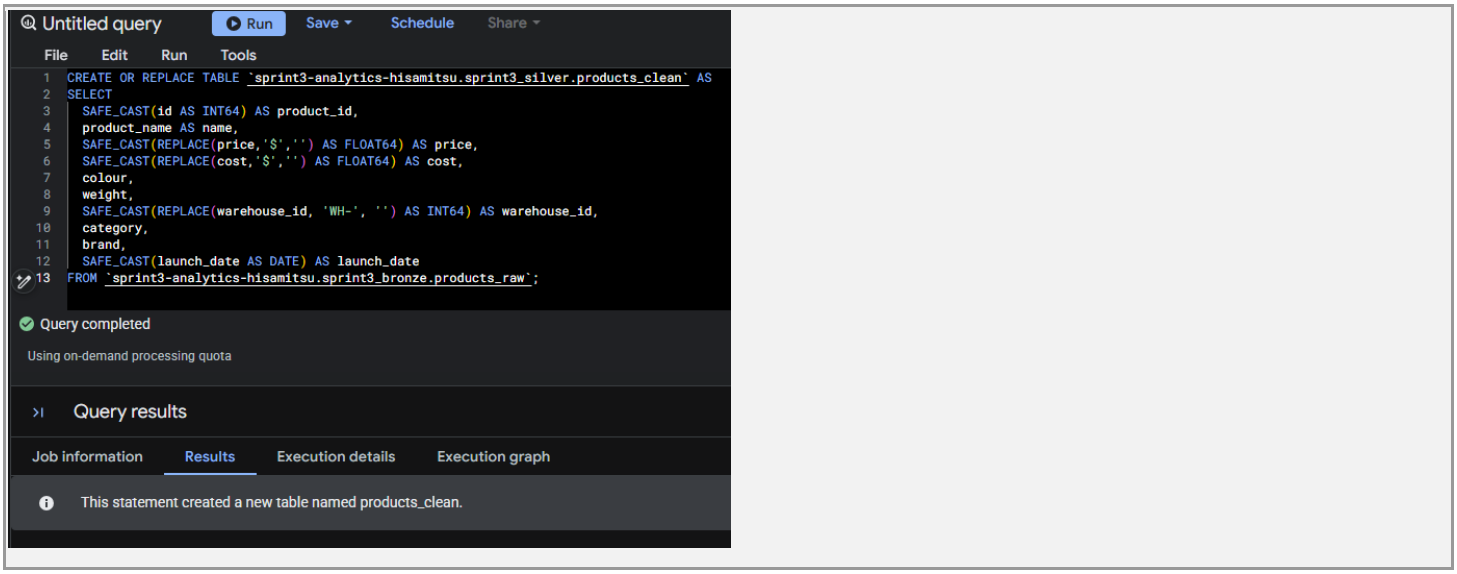
NIVEL 2: Limpieza y transformacion (ELT)

Ejercicio 1 – Limpieza de productos

Enunciado

Crear sprint3_silver.products_clean a partir de products_raw: renombrar id a product_id y product_name a name, limpiar warehouse_id (quitar prefijo WH- y convertir a INT64) y asegurar price como FLOAT64 sin simbolos de moneda.

Evidencia de ejecución



Comentarios sobre los resultados

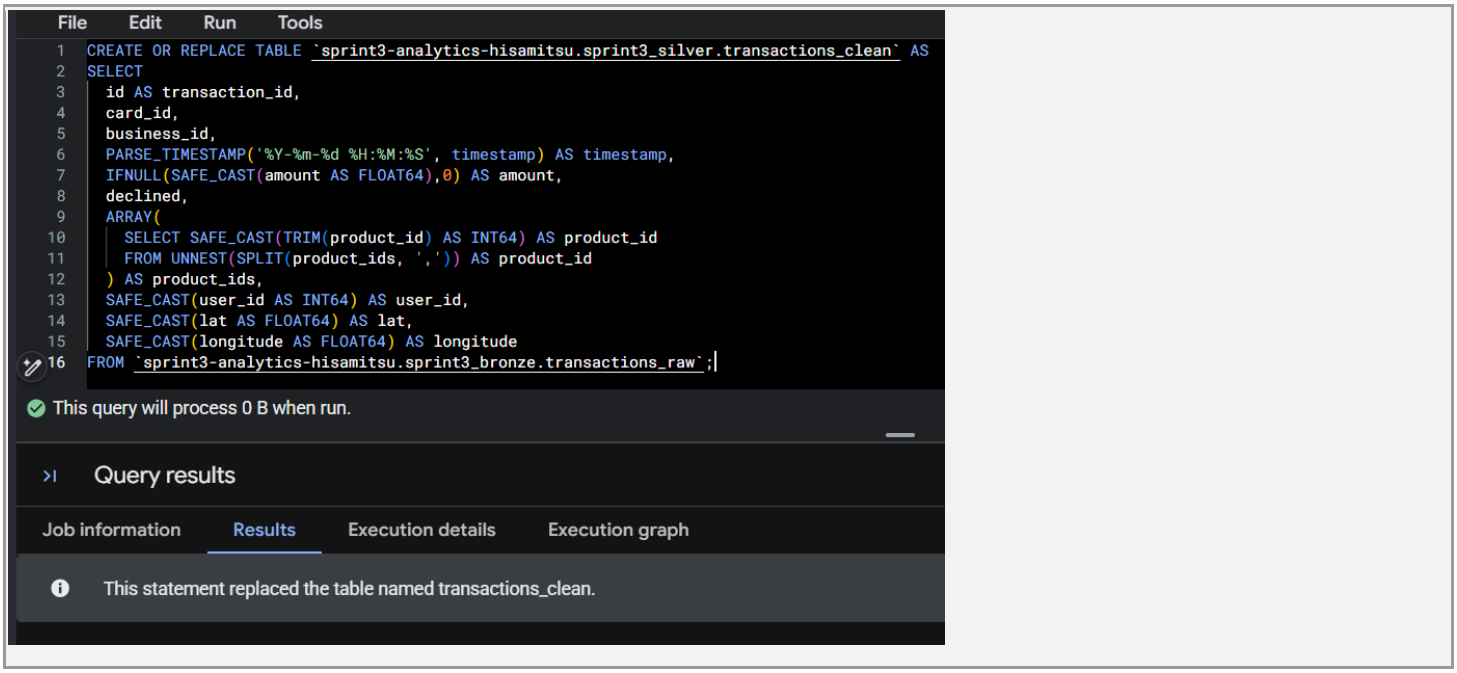
products_clean creada en la capa Silver. El SAFE_CAST de Price y cost lo hice simple, con REPLACE de '$'. Podría usar una expresión regular para evitar errores con otros caracteres no numéricos.

Ejercicio 2 – Creacion de transacciones limpias (capa Silver)

Enunciado

Crear sprint3_silver.transactions_clean: renombrar id a transaction_id, SAFE_CAST + IFNULL en amount, timestamp real, lat/longitude en FLOAT64 y product_ids como ARRAY<INT64>.

Evidencia de ejecución

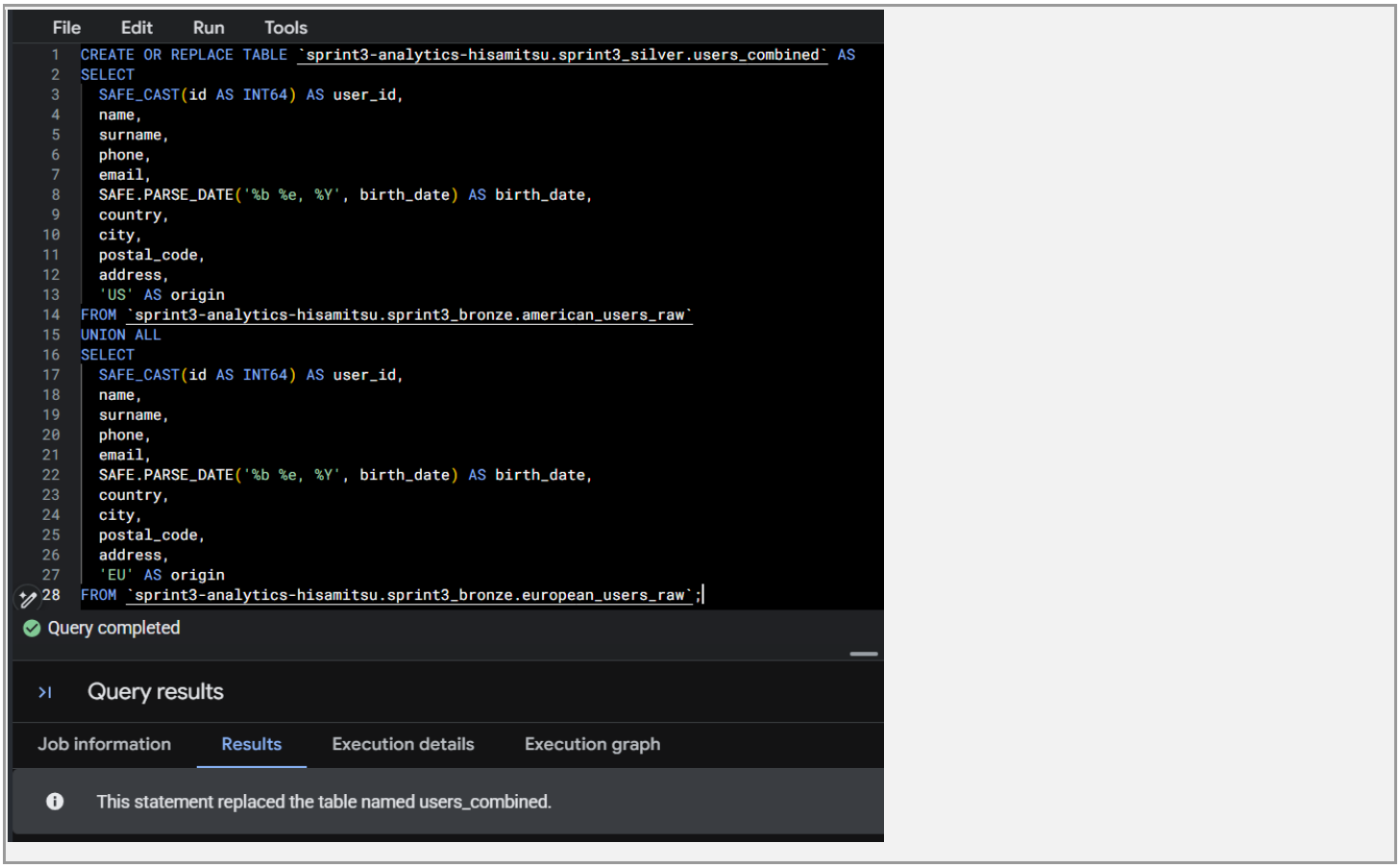


Ejercicio 3 – Unificación de usuarios (UNION)

Enunciado

Crear sprint3_silver.users_combined uniendo american_users_raw y european_users_raw con UNION ALL, renombrando id a user_id y anadiendo la columna origin.

Evidencia de ejecución

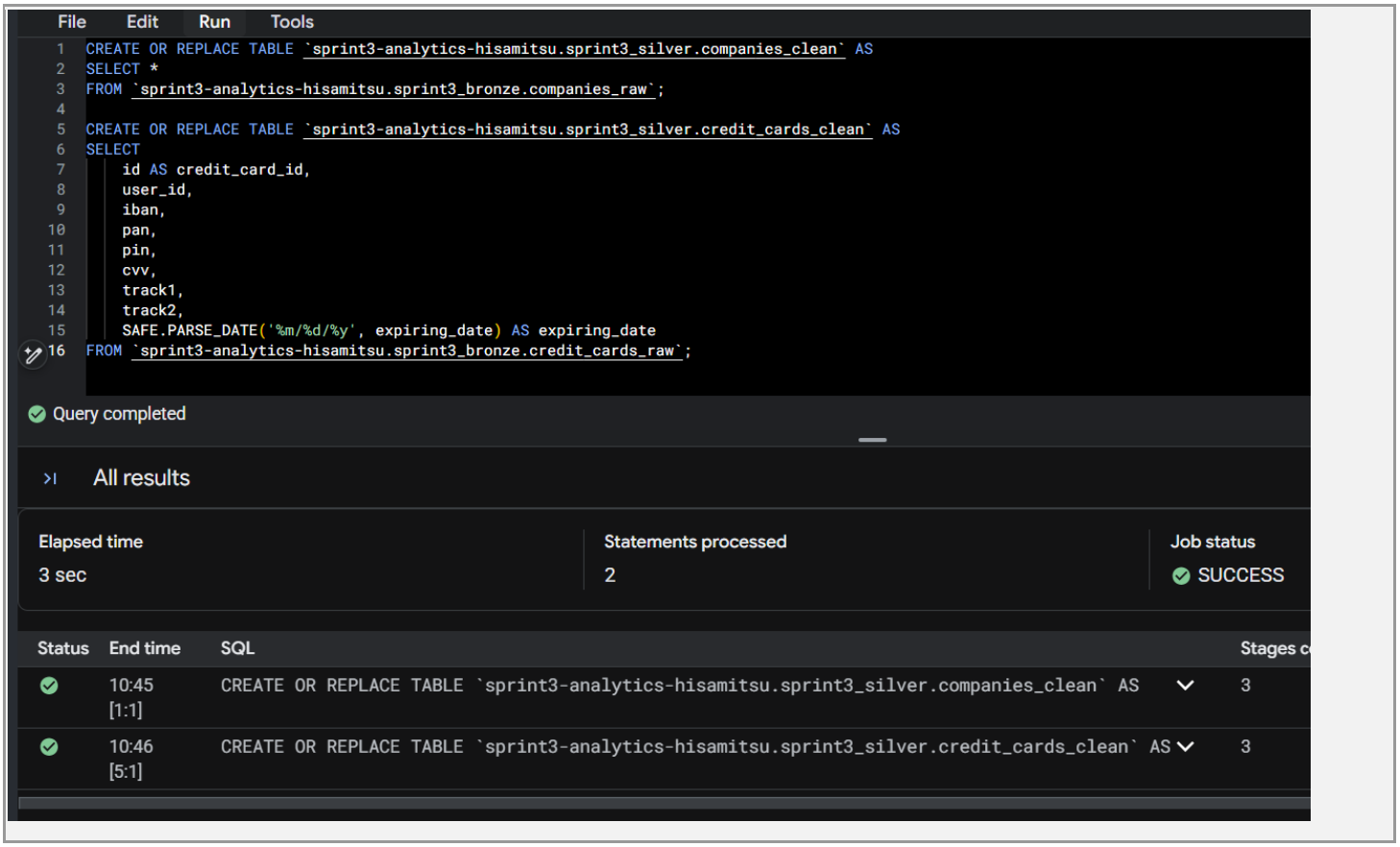


Ejercicio 4 – Materialización de compañías y tarjetas de crédito

Enunciado

Crear sprint3_silver.companies_clean y sprint3_silver.credit_cards_clean como tablas nativas (copia directa desde las tablas externas, renombrando el id si corresponde).

Evidencia de ejecución



NIVEL 3: Presentacion de datos y creacion de vistas

Ejercicio 1 – La vista de marketing

Enunciado

*Crear sprint3_gold.v_marketing_kpis con nombre, telefono y pais de cada compania, media de compra (AVG(amount)) y la columna calculada client_tier (Premium si la media es superior a 260 euros, si no Standard). Entrega: SELECT * sobre la vista ordenado por Premium primero y, dentro de cada tier, por media de compra descendente.*

Evidencia de ejecución

File Edit Run Tools

```sql
CREATE OR REPLACE VIEW `sprint3-analytics-hisamitsu.sprint3_gold.v_marketing_kpis` AS
SELECT
  company.company_name,
  company.phone,
  company.country,
  ROUND(AVG(transactions.amount),2) AS average_amount,
  IF(AVG(transactions.amount)>260,'Premium','Standard') AS client_tier
FROM `sprint3-analytics-hisamitsu.sprint3_silver.companies_clean` AS company
INNER JOIN `sprint3-analytics-hisamitsu.sprint3_silver.transactions_clean` AS transactions
  ON company.company_id = transactions.business_id
GROUP BY company.company_name, company.phone, company.country;

SELECT *
FROM `sprint3-analytics-hisamitsu.sprint3_gold.v_marketing_kpis`
ORDER BY average_amount DESC;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	company_name	phone	country	average_amount	client_tier
1	Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87	Premium
2	Pretium Neque Corp.	07 77 48 55 28	Australia	276.16	Premium
3	Urna Convallis Associates	06 01 24 77 04	United States	274.24	Premium
4	At Associates	09 56 61 10 65	New Zealand	272.21	Premium
5	Metus Vitae Associates	08 25 44 40 66	Australia	270.08	Premium
6	Aliquet Diam Limited	02 76 61 47 46	United States	269.6	Premium
7	Nec Luctus LLC	02 14 71 75 73	Norway	268.6	Premium

Comentarios sobre los resultados

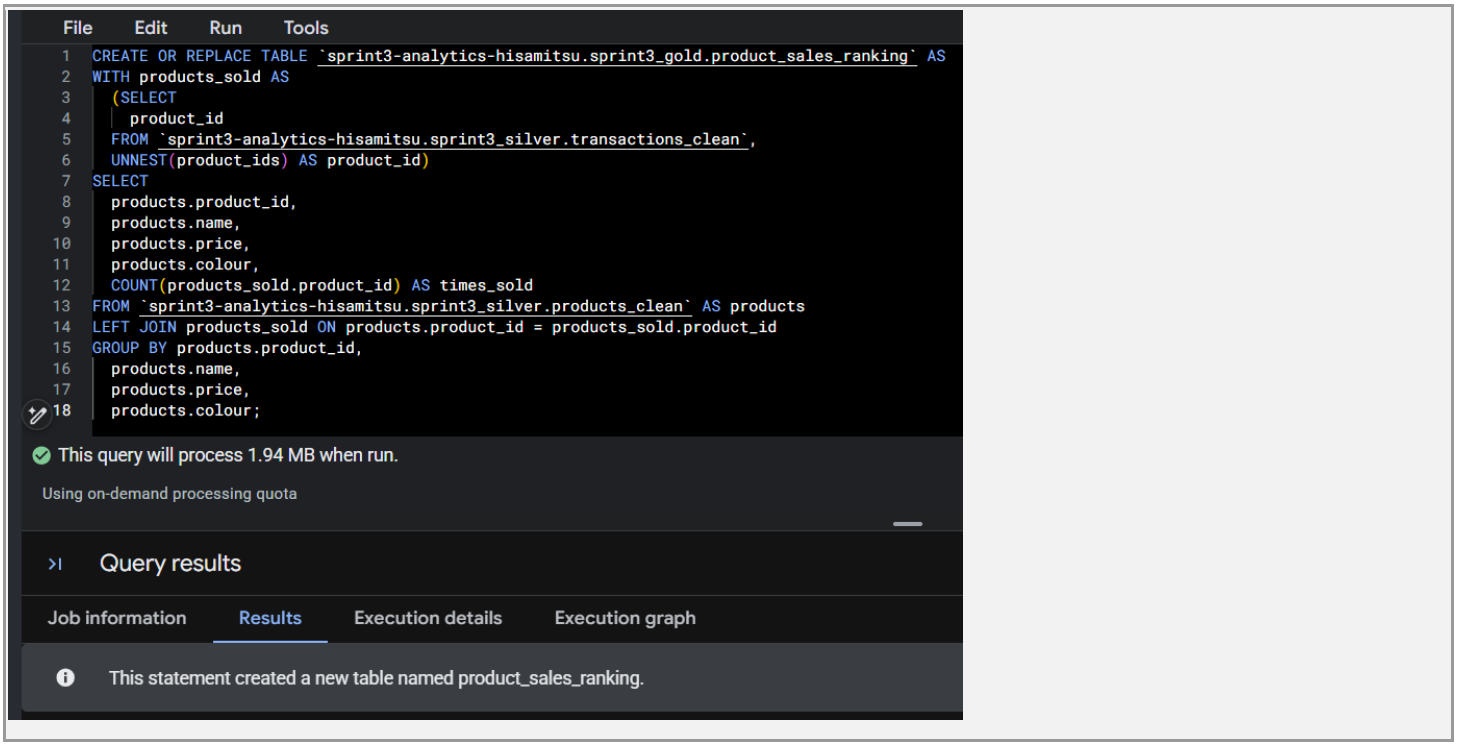
No hace falta ordenar por client_tier pues ordenar solo por average_amount DESC ya deja Premium primero.

Ejercicio 2 – Ranking de productos (arrays)

Enunciado

Crear sprint3_gold.product_sales_ranking con product_id, name, price y colour, mas total_sold (numero de veces que aparece el producto en las transacciones, usando UNNEST sobre product_ids). LEFT JOIN desde products_clean para no perder los productos con 0 ventas.

Evidencia de ejecución

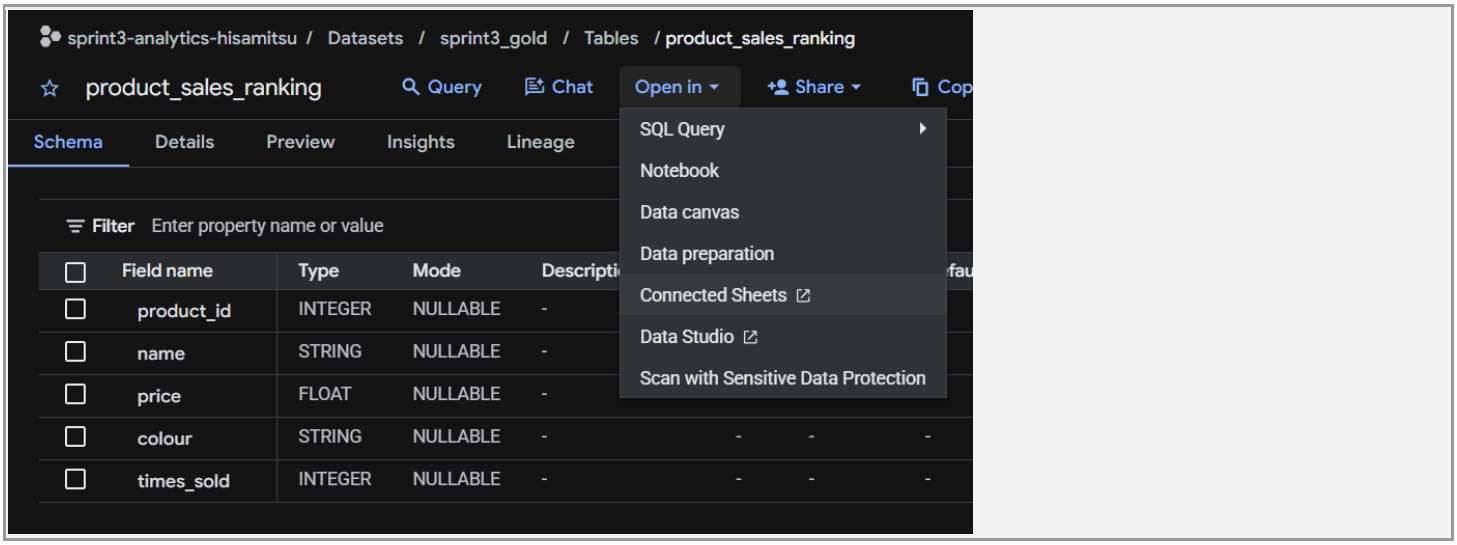


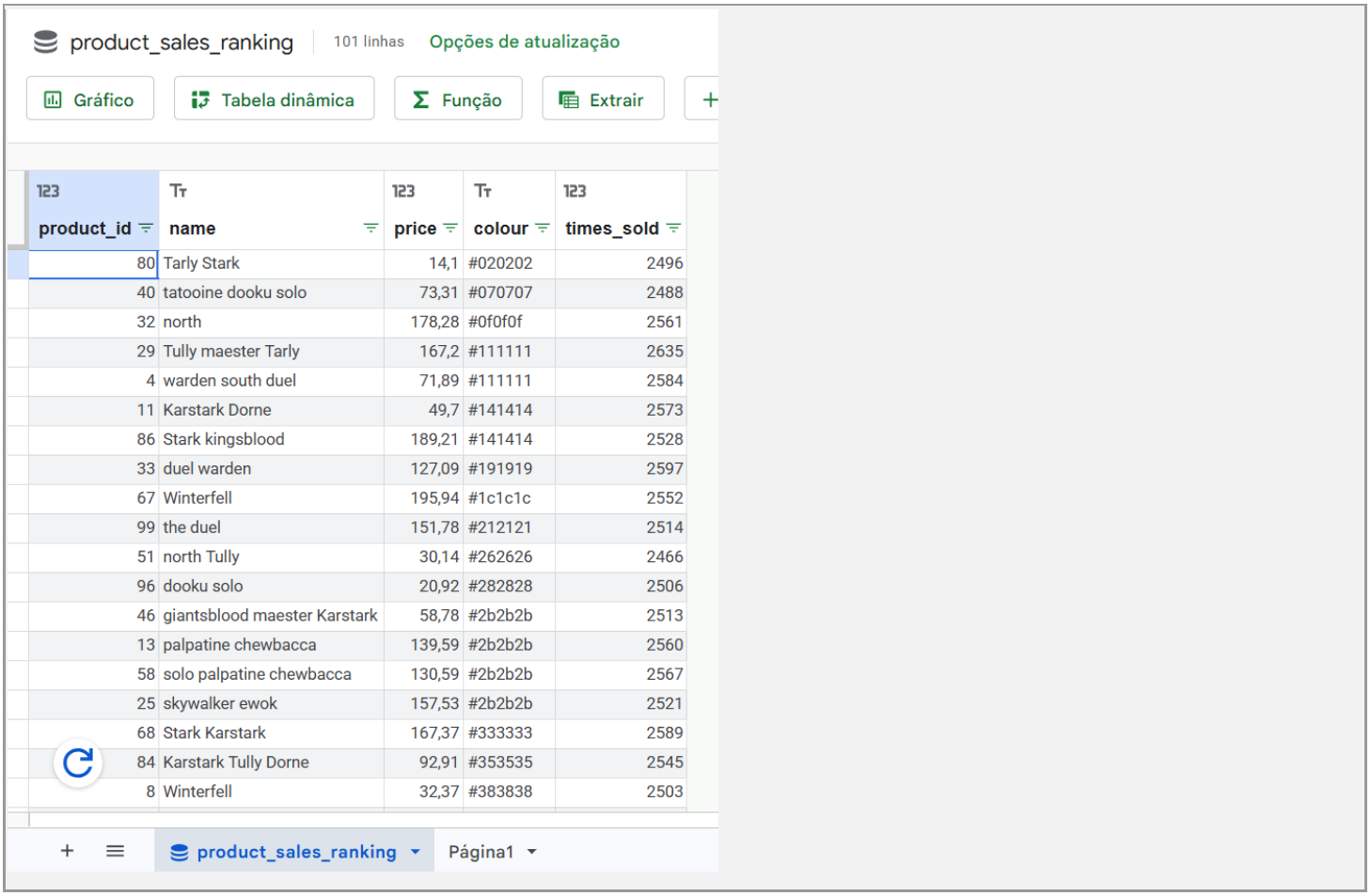
Ejercicio 3 – Exportacion de resultados

Enunciado

Exportar product_sales_ranking a Google Sheets o descargarlo como CSV para el informe del manager.

Evidencia de ejecución





product_id	name	price	colour	times_sold
80	Tarly Stark	14,1	#020202	2496
40	tatooine dooku solo	73,31	#070707	2488
32	north	178,28	#0f0f0f	2561
29	Tully maester Tarly	167,2	#111111	2635
4	warden south duel	71,89	#111111	2584
11	Karstark Dorne	49,7	#141414	2573
86	Stark kingsblood	189,21	#141414	2528
33	duel warden	127,09	#191919	2597
67	Winterfell	195,94	#1c1c1c	2552
99	the duel	151,78	#212121	2514
51	north Tully	30,14	#262626	2466
96	dooku solo	20,92	#282828	2506
46	giantsblood maester Karstark	58,78	#2b2b2b	2513
13	palpatine chewbacca	139,59	#2b2b2b	2560
58	solo palpatine chewbacca	130,59	#2b2b2b	2567
25	skywalker ewok	157,53	#2b2b2b	2521
68	Stark Karstark	167,37	#333333	2589
84	Karstark Tully Dorne	92,91	#353535	2545
8	Winterfell	32,37	#383838	2503